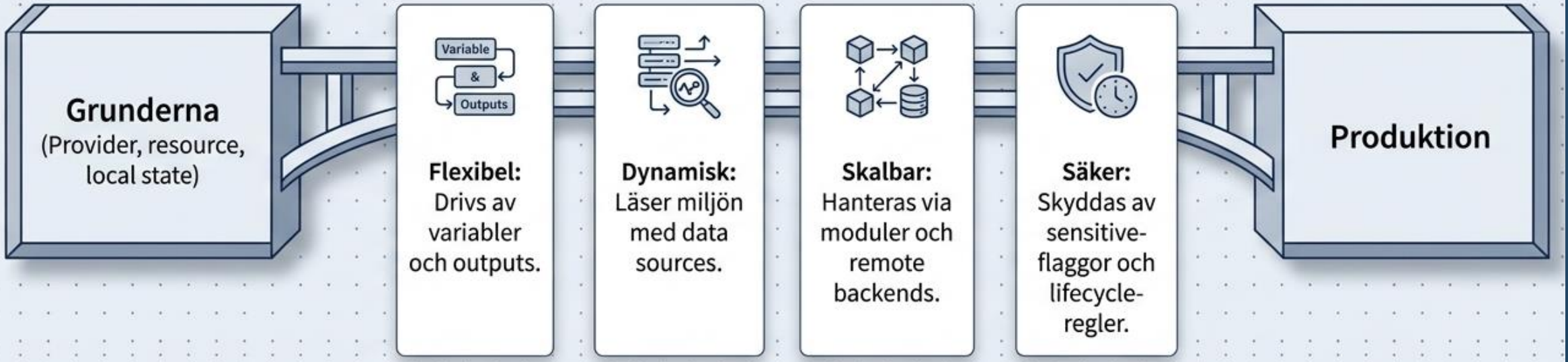
Terraform Fortsättning

Arseniy Buzyan

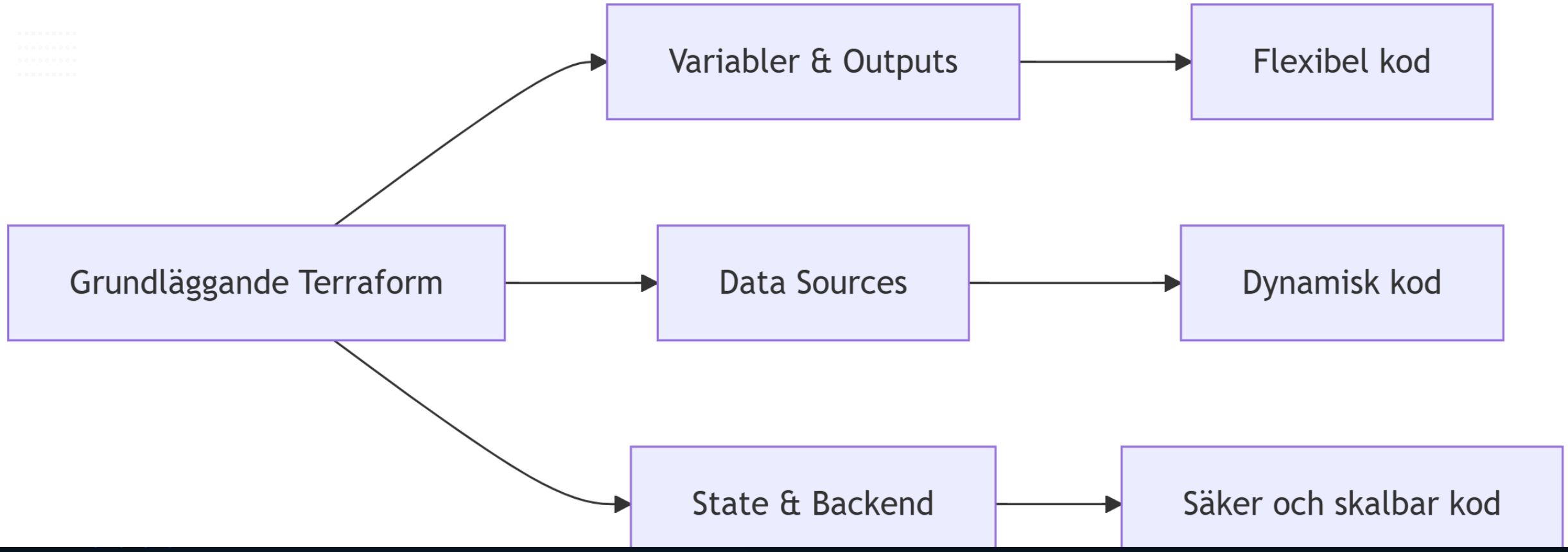
Steget från det lokala till produktion



Svenska arbetsgivare förväntar sig **återanvändbara** och **säkra konfigurationer.** Enligt Scalr är det **variabler** och **outputs** som gör din kod återanvändbar mellan olika miljöer.

Diagram – resan från grund till avancerad

Varför detta är viktigt: Svenska arbetsgivare förväntar sig att du kan skriva återanvändbara, flexibla och säkra Terraform-konfigurationer – inte bara enkla resurser. Enligt Scalr är variabler och outputs det som gör Terraform-konfigurationer återanvändbara mellan olika miljöer och deployments .



Från Verkligheten – Terraform I Svenska Företag

- Terraform används i svenska företag för att hantera allt från molninfrastruktur till lokala konfigurationer.
- Spotify Provisionerar molninfrastruktur med Terraform och använder moduler för återanvändbarhet
- Klarna Hanterar serverkonfiguration och nätverk med Terraform och Ansible
- Volvo Automatiserar driftsättning i hybridmiljöer med Terraform

- Varför detta är viktigt: Företag använder Terraform för att automatisera repetitiva uppgifter, minska manuella fel och spara tusentals arbetstimmar. Enligt Scalr är Terraform state-filen en JSON-fil som kartlägger din konfiguration till verkliga resurser och håller reda på varje resurs och dess attribut .

Variabler – Gör Din Kod Flexibel

Input-variabler = Gör din konfiguration återanvändbar

Problem – hårdkodade värden

```hcl
# ❌ Hårdkodat – kan bara användas för WEB01!
resource "local_file" "config" {
  filename = "C:/temp/WEB01_config.json"
  content  = "server=WEB01"
}
```

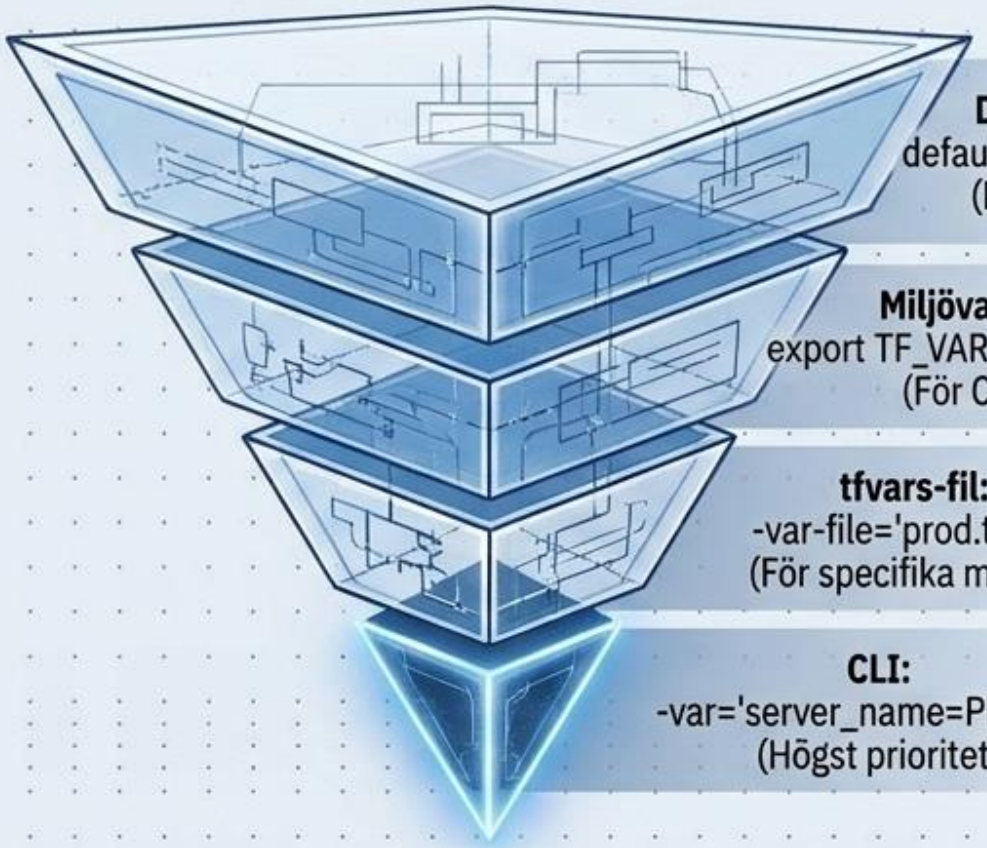
En variabels anatomi och livscykel

```hcl
variable "server_name" {
  type        = string
  description = "Namnet på servern"
  default     = "WEB01"
}
```

Anger datatyp: string, number, bool

Dokumenterar syftet

Gör variabeln valfri



Default:
default = 'WEB01'
(Enklast)

Miljövariabel:
export TF_VAR_server_name
(För CI/CD)

tfvars-fil:
-var-file='prod.tfvars'
(För specifika miljöer)

CLI:
-var='server_name=PROD01'
(Högst prioritet)

Enligt **Scalr:** Input-variabler är som funktionsargument – de parametriserar din konfiguration.

Lösning – variabler

```
# ✅ Flexibelt – kan användas för vilken server som helst!
variable "server_name" {
  type        = string
  description = "Namnet på servern"
  default     = "WEB01"
}

resource "local_file" "config" {
  filename = "C:/temp/${var.server_name}_config.json"
  content  = "server=${var.server_name}"
}
```

Viktiga attribut för variabler

Enligt Scalr: Input-variabler är som funktionsargument – de parametriserar dina konfigurationer så att de blir återanvändbara mellan olika miljöer och deployments.

Attribut	Vad gör den?	Exempel
type	Anger datatyp (string, number, bool, list, map)	type = string
description	Förklarar vad variabeln används till	description = "Namnet på servern"
default	Ger ett standardvärde (gör variabeln valfri)	default = "WEB01"
sensitive	Döljer värdet i output och loggar	sensitive = true

Så Här Använder Du Variabler

Du kan ange variabelvärden på flera sätt – här är de vanligaste metoderna:

Metod	Exempel	När används?
Default	default = "WEB01"	Standardvärden – enklast
Kommandoraden	terraform apply -var="server_name=PROD01"	Snabba tester och engångskörningar
Miljövariabel	export TF_VAR_server_name="PROD01"	CI/CD-pipelines
tfvars-fil	terraform apply -var-file="prod.tfvars"	Olika miljöer (dev, test, prod)

Exempel – tfvars-fil för produktion (prod.tfvars)

- hcl
- ```
  # prod.tfvars – specifika värden för produktionsmiljön
  ```
- ```
  server_name    = "PROD01"
  ```
- ```
  environment    = "production"
  ```
- ```
  instance_count = 5
  ```
- **Kör med rätt miljö:**
- bash
- ```
  terraform apply -var-file="prod.tfvars"
  ```
- **Viktigt att veta:** Enligt Scalr kan variabler tillhandahållas genom flera metoder: kommandoraden, miljövariabler, interaktiva uppmaningar (om inget värde anges), default-värden, eller via tfvars-filer .
  ```

Validera Variabler – Fånga Fel Tidigt

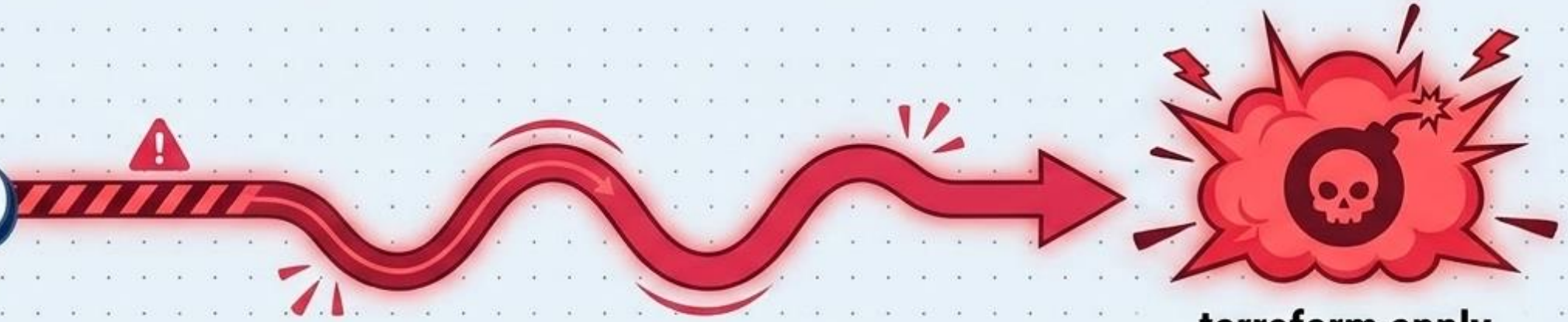
validation-block låter dig kontrollera att variabler har korrekta värden

Fånga fel innan de kostar pengar

Utan validering

terraform plan

(Ogiltiga värden accepteras)



terraform apply

(Fel upptäcks när resurser skapas – dyrt och svårt att felsöka!)

Med validering

terraform plan



(Fel upptäcks direkt, tydliga felmeddelanden)

Safety Net

```
validation {
  condition = contains(["dev", "prod"], var.environment)
  error_message = "Måste vara dev eller prod."
}
```

Exempel – validera miljö och tal:

```hcl
variable "environment" {
  type        = string
  description = "Miljö: dev, test, staging, production"

  validation {
    condition     = contains(["dev", "test", "staging", "production"], var.environment)
    error_message = "Environment måste vara dev, test, staging eller production."
  }
}

variable "instance_count" {
  type        = number
  description = "Antal servrar att skapa"

  validation {
    condition     = var.instance_count > 0 && var.instance_count <= 10
```

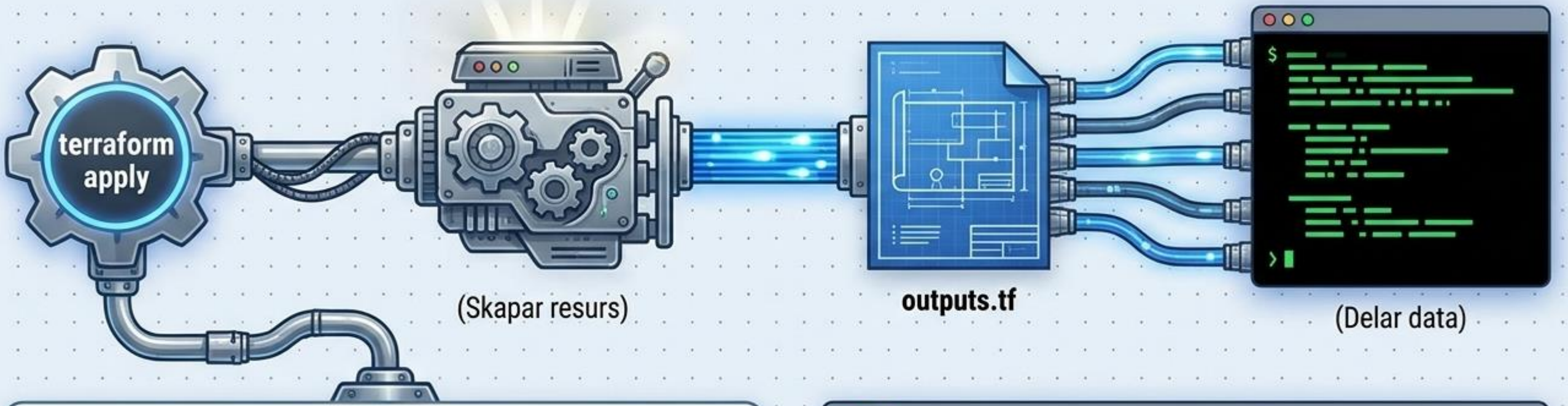
Varför är detta viktigt?

Utan validering	Med validering
Fel värden upptäcks först när resurser skapas (dyrt!)	Fel upptäcks direkt vid terraform plan
Kan skapa resurser med felaktiga inställningar	Ogiltiga värden accepteras inte
Svårare att felsöka varför något gick fel	Tydliga felmeddelanden

Outputs – Visa Resultat och Dela Data

Outputs visar information efter terraform apply **– användbara för att dela data mellan moduler**

Outputs som systemets blodomlopp



The Connector

```
output "file_path" {
  description = "Sökväg till den skapade filen"
  value       = local_file.hello.filename
}
```

```
> terraform output file_path
> terraform output -json
>
```

Enligt **Scalr** kan du fråga outputs när som helst efter en apply.

Exempel – outputs:

```
resource "local_file" "hello" {
  filename = "C:/temp/hello.txt"
  content  = "Hej världen!"
}

# outputs.tf
output "file_path" {
  description = "Sökväg till den skapade filen"
  value       = local_file.hello.filename
}

output "file_content" {
  description = "Innehållet i filen"
  value       = local_file.hello.content
}

output "sensitive_password" {
  description = "Databaslösenord"
  value       = var.db_password
```

Hämta outputs efter `apply`:

Viktigt att veta: Enligt Scalr visas outputs efter `terraform apply`. Du kan fråga dem senare med `terraform output`.

bash

```
# Visa alla outputs
terraform output

# Visa en specifik output
terraform output file_path

# Hämta JSON-format
terraform output -json
```

Data Sources – Läs Befintlig Information

- Data sources = "Läs bara" – hämtar information utan att skapa eller ändra något.
- **Vad är en data source?**
  En data source är ett sätt för Terraform att läsa information från externa källor – t.ex. filer, moln-API:er eller andra Terraform-states. Den skapar inga resurser, bara läser och gör data tillgänglig i din konfiguration.

Att läsa verkligheten med Data Sources



data ‘local_file’ ‘policy’

Läs filen/miljön
(Read-only data extraction)

Data används i resurser
(Injected into infrastructure)

Resource	Data Source
Skapar, ändrar, raderar (”Läs och Skriv”).	Hämtar befintlig information utan att ändra (”Bara Läs”). Exempel: local_file, terraform_remote_state, http.

Läsa en befintlig fil

hcl

```hcl
# data.tf - Läs en JSON-fil
data "local_file" "policy" {
  filename = "${path.module}/policy.json"
}

# Använd innehållet i en resource
resource "local_file" "deployment_ready" {
  filename = "C:/temp/deployment_ready.txt"
  content  = data.local_file.policy.content
# Innehållet från filen
}
```

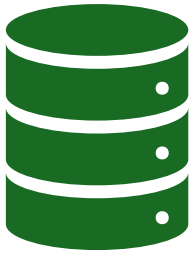
Vanliga data sources



local_file Läsa filer från din lokala dator



terraform_remote_state Läsa outputs från en annan Terraform-state



http Hämta data från en HTTP-endpoint

Terraform State – Ditt Viktigaste "Minne"

State = Terraforms minne – håller reda på vad som har skapats

Vilka resurser finns? local_file.config

Vilka ID:n har de? Filens sökväg

Vilka attribut? Filens innehåll

Metadata Tidsstämplar, versioner

State finns i filen terraform.tfstate

State-filen: Systemets minne och hjärna



Vad finns i hjärnan?

- **Resurs-ID:n** (Kopplar din kod till faktiska servrar)
- **Attribut** (T.ex. filens exakta innehåll)
- **Metadata** (Tidsstämplar, versioner)

Enligt Scalr innehåller state-filen känslig information och kartlägger din konfiguration till verkliga resurser. Den måste skyddas.

Riskerna med lokal State

Säkerhetsrisken



Fara: Känslig information (lösenord, API-nycklar) lagras i klartext på din dator.



Lösning:
Markera med sensitive = true och kryptera.

Samarbetsrisken



Fara: Lokal state är inte delbar. Teammedlemmar kan skriva över varandras ändringar.



Lösning:
Remote backend med State-locking.

Katastrofrisken



Fara: Filen kan förloras eller korrumperas om en hårddisk kraschar.



Lösning:
Molnlagring med versioning (t.ex. S3).

Enterprise-standarden: Remote Backend

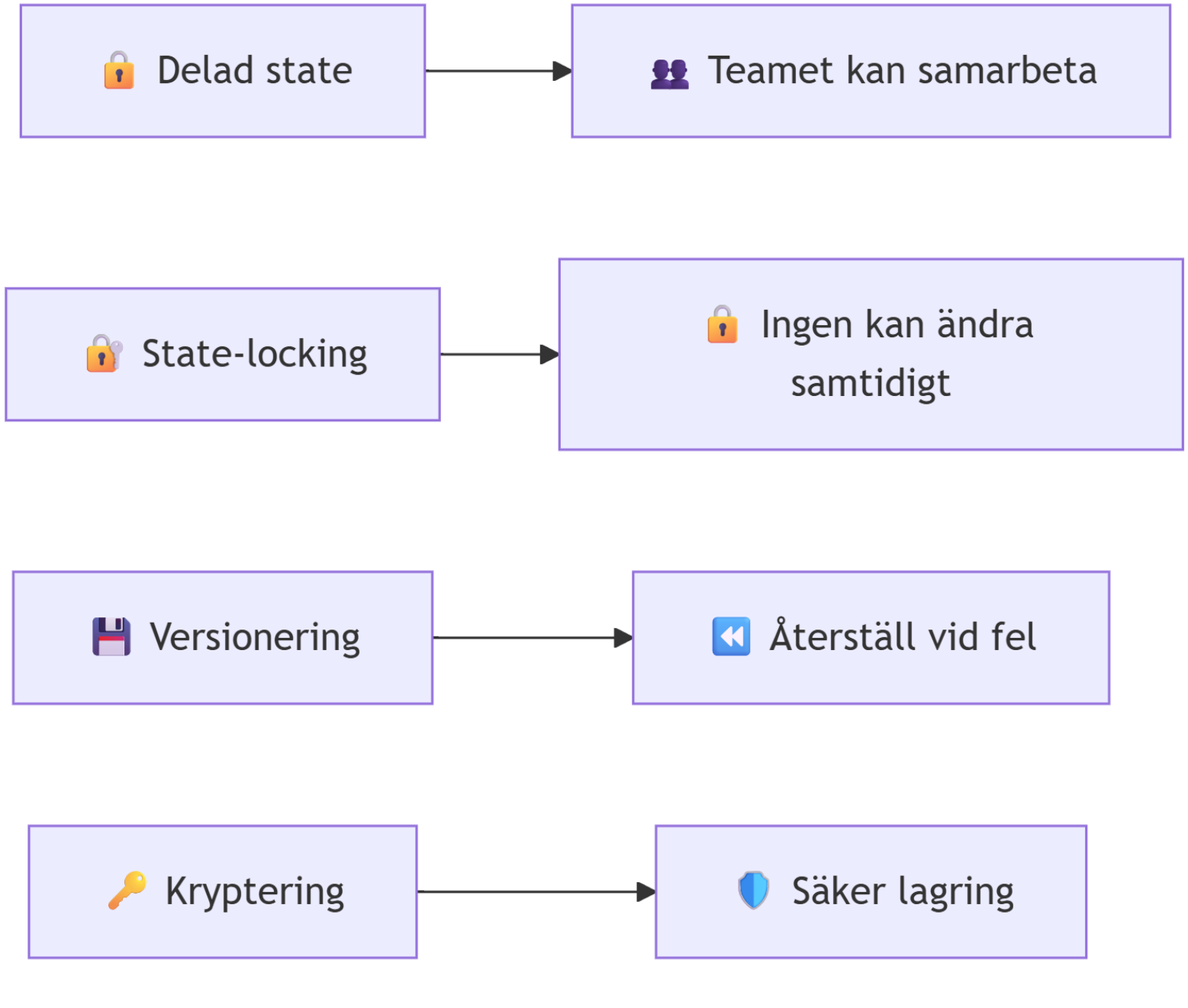


Code Blueprint

```
backend "s3" {
  bucket         = "my-terraform-state"
  region         = "eu-north-1"
  encrypt        = true
  dynamodb_table = "terraform-locks"
}
```

- Delad åtkomst för teamet
- State-locking förhindrar kollisioner
- Kryptering för säkerhet
- Versionshantering för återställning

Varför remote backend?



Diagnos: Local vs. Remote State

	Local	Remote
Delning (Samarbete)	Svårt – filen är låst till en dator.	Enkelt – hela teamet delar en “Single Source of Truth”.
State-låsning (Locking)	Finns inte (Risk för korrupt data).	**Ja** – DynamoDB förhindrar att två kör apply samtidigt.
Säkerhet & Backup	Hög risk vid hårdvarufel, hemligheter i klartext.	**Krypterat** i vila, inbyggd versioning i molnet.

Exempel – S3 backend (AWS)

- Enligt Scalr: Remote backend rekommenderas för team då det ger delad åtkomst, state-låsning för att förhindra samtidiga ändringar, kryptering i vila, och versionshantering .
  - hcl
- ```
  terraform {
    backend "s3" {
      bucket         = "my-terraform-state"
      key            = "prod/terraform.tfstate"
      region         = "eu-north-1"
      encrypt        = true
      dynamodb_table = "terraform-locks"  # State-locking
    }
  }
  ```
  ```

Lifecycle-regler skyddar viktiga resurser från oavsiktlig radering

Lifecycle-regler – Skydda Viktiga Resurser



Exempel – förhindra radering av databas

```hcl
resource "local_file" "important_config" {
  filename = "C:/prod/database_config.json"
  content  = "secret=value"

  lifecycle {
    prevent_destroy = true   # Terraform kan inte ta bort denna fil!
  }
}
```

Exempel – skapa ny resurs innan gammal tas bort (noll driftstopp)

```hcl
resource "aws_instance" "web" {
  ami           = "ami-123456"
  instance_type = "t3.medium"

  lifecycle {
    create_before_destroy = true
# Ny skapas innan gammal tas bort
  }
}
```

Immunsystemet: Lifecycle-regler

Skölden (Prevent Destroy)



Use Case: Databaser, lagring, kritiska resurser.

```
prevent_destroy = true
```

(Terraform vägrar ta bort resursen, även vid felaktig apply).

Bron (Create Before Destroy)



Use Case: Noll driftstopp (Zero downtime) vid uppdateringar.

```
create_before_destroy = true
```

Workshop – Skapa En Katalogstruktur Med Terraform

Mål: Använda Terraform, variabler och outputs för att skapa en komplett katalogstruktur.

Skapa main.tf

```hcl
provider "local" { }

resource "local_file" "root_folder" {
  filename = "C:/Projekt/${var.project_name}/.keep"
  content  = "# Projekt: ${var.project_name}\n# Miljö: ${var.environment}"
}

output "project_path" {
```

Skapa variables.tf

```hcl
variable "project_name" {
  type        = string
  description = "Namnet på projektet"
  default     = "mittprojekt"
}

variable "environment" {
  type        = string
  description = "Miljö: dev, test, staging, production"
```

Steg 3 – Kör Terraform

- `bash`
- `terraform init`
- `terraform plan`
- `terraform apply -auto-approve`

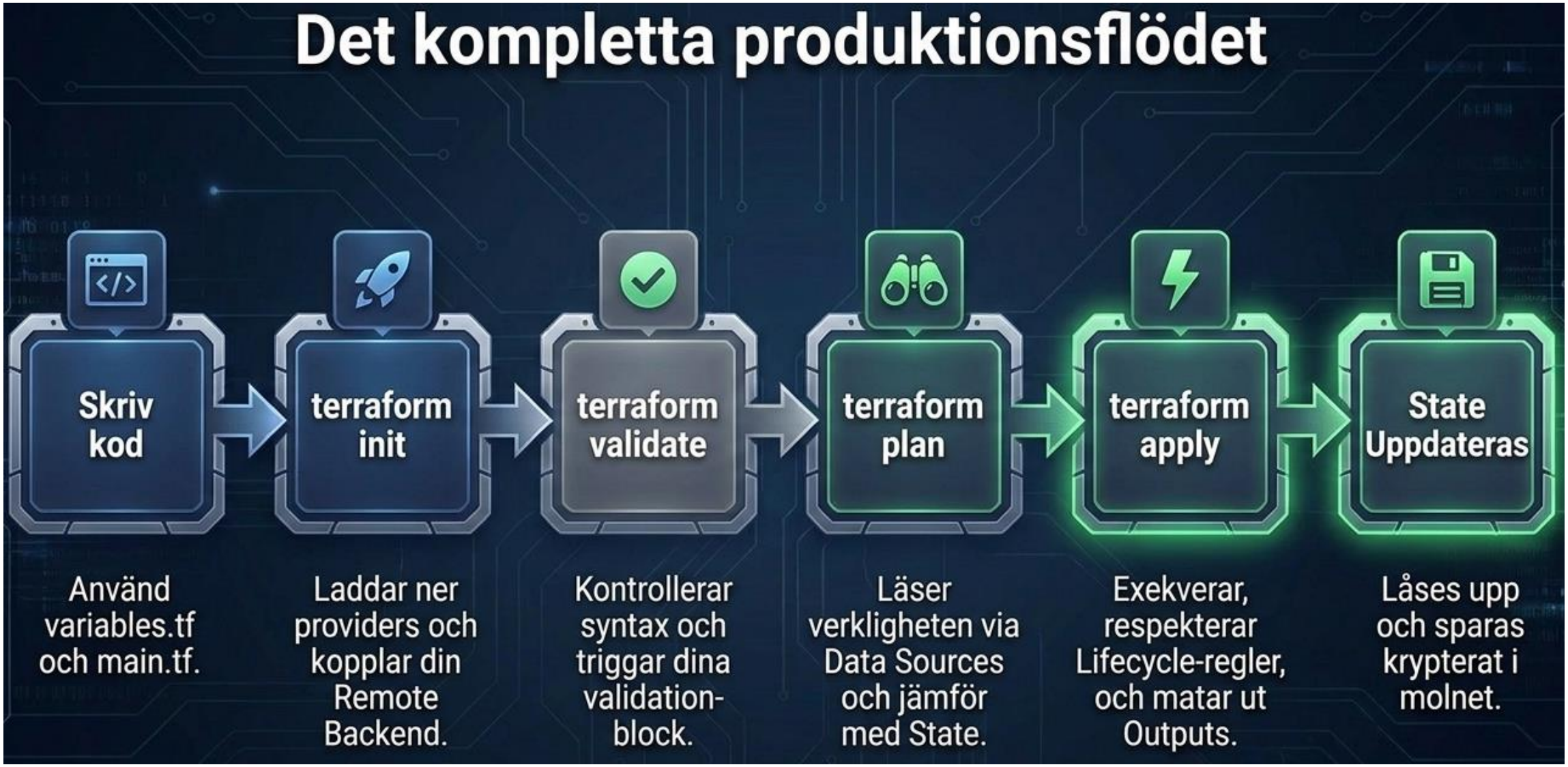
Kontrollera att struktur skapades

bash

dir C:/Projekt/mittprojekt

Diskussionsfråga: Varför är det bra att Terraform kan jämföra state med verkligheten innan den gör ändringar? Varför är detta bättre än att bara köra ett skript?

Det kompletta produktionsflödet



Katalogstrukturen i praktiken

C:/Projekt/mittprojekt

- main.tf (Logiken: Providers och resurser)
- variables.tf (Definitioner: Typer, default-värden, validering)
- outputs.tf (Resultat: Vad vi vill exportera)
- prod.tfvars (Miljöspecifika värden för produktion)



Att separera koden i dessa filer är inte ett tekniskt krav, utan ett arkitektoniskt mönster för att maximera läsbarhet och återanvändbarhet i team.

Vad Har Vi Lärt Oss Idag?

Variabler: Gör din kod flexibel och återanvändbar med variable-block

Outputs: Visa vad du har skapat med output-block

Data sources: Läs befintlig information från filer eller andra källor

Terraform State: Terraforms "minne" – håller reda på skapade resurser

Remote Backend: Dela state med teamet, state-locking och kryptering

Lifecycle-regler: Skydda viktiga resurser från oavsiktlig radering

Quiz:

https://forms.gle/1Wdjq1CJ39LM2TA8A